

Desafio de Quebra de Senha — Estudo de Paradigmas Algorítmicos

Relatório Técnico Experimental

Autor

Luiz Gustavo Dall'Agnol

Resumo

Este trabalho apresenta a implementação e análise comparativa de quatro paradigmas de projeto de algoritmos aplicados ao problema de quebra de senhas alfanuméricas de oito caracteres. Foram desenvolvidas soluções utilizando Força Bruta, Divisão e Conquista, Programação Dinâmica e Algoritmos Genéticos, todas integradas a um sistema de validação denominado “Oráculo”. O objetivo principal foi avaliar o comportamento de cada paradigma em relação ao tempo de execução, número de consultas ao oráculo e consumo de memória. Os resultados demonstram que a quantidade de informação fornecida pelo sistema de validação altera drasticamente a complexidade do problema, tornando abordagens heurísticas e estratégias de decomposição significativamente mais eficientes do que a busca exaustiva tradicional.

1. Introdução

Na área de Segurança da Informação, ataques de força bruta são frequentemente utilizados em testes de intrusão para verificar a robustez de sistemas de autenticação. Entretanto, sistemas modernos geralmente possuem mecanismos de proteção contra tentativas excessivas de acesso, como bloqueios temporários, limitação de taxa de requisições e autenticação multifatorial.

O presente trabalho propõe um ambiente controlado para estudo de paradigmas algorítmicos aplicados à quebra de senhas. Em vez de apenas validar se uma senha está correta, o sistema utilizado fornece diferentes níveis de feedback para os algoritmos, permitindo explorar estratégias mais sofisticadas que a simples tentativa exaustiva.

O desafio consiste em descobrir uma senha alfanumérica de oito caracteres pertencente ao conjunto:

- Letras minúsculas (a-z)
- Letras maiúsculas (A-Z)
- Números (0-9)

O espaço de busca total do problema é dado por:

$62^8 \approx 2,18 \times 10^{14}$ combinações

Esse valor demonstra que uma abordagem puramente exaustiva torna-se impraticável em cenários reais.

2. Objetivos

Os principais objetivos deste trabalho foram:

1. Implementar quatro paradigmas distintos de resolução do problema.
 2. Comparar desempenho computacional entre as estratégias.
 3. Avaliar o impacto do tipo de feedback fornecido pelo sistema de validação.
 4. Analisar o custo computacional em termos de tempo, memória e número de consultas ao oráculo.
 5. Discutir a aplicabilidade prática de cada abordagem em sistemas reais.
-

3. Fundamentação Teórica

3.1 Força Bruta

A estratégia de força bruta consiste em testar sistematicamente todas as combinações possíveis até encontrar a senha correta. Trata-se da abordagem mais simples e direta, porém apresenta crescimento exponencial do custo computacional.

Sua complexidade temporal é:

$$O(62^8)$$

A principal vantagem desta abordagem é sua garantia de encontrar a solução. Entretanto, seu custo cresce rapidamente com o tamanho da senha.

3.2 Divisão e Conquista

O paradigma de Divisão e Conquista consiste em decompor um problema maior em subproblemas menores e independentes.

Neste trabalho, a senha de 8 caracteres foi dividida em dois blocos de 4 caracteres. Utilizando a função `check_partial()`, tornou-se possível validar cada metade separadamente.

A complexidade aproximada passa a ser:

$$O(2 \times 62^4)$$

Essa redução representa uma queda extremamente significativa no espaço de busca.

3.3 Programação Dinâmica

A Programação Dinâmica busca evitar recomputações por meio do armazenamento de resultados intermediários.

Neste projeto, foi utilizada memoização de prefixos válidos e inválidos. Quando um prefixo falhava, qualquer senha derivada dele era automaticamente descartada.

Essa estratégia reduz drasticamente consultas redundantes ao sistema de validação.

3.4 Algoritmos Genéticos

Algoritmos Genéticos são técnicas heurísticas inspiradas no processo de seleção natural.

Cada senha foi tratada como um cromossomo de 8 genes. O algoritmo utilizou:

- População inicial aleatória
- Avaliação de fitness
- Seleção natural
- Cruzamento
- Mutação

A função de fitness retornava quantos caracteres estavam corretos na posição correta.

Essa informação guiou a evolução progressiva das soluções.

4. Metodologia

4.1 Linguagem Utilizada

O projeto foi implementado em Python utilizando o Visual Studio Code como ambiente de desenvolvimento.

A escolha da linguagem ocorreu devido:

- Facilidade de prototipagem
 - Manipulação eficiente de strings
 - Bibliotecas nativas para geração combinatória
 - Rapidez no desenvolvimento experimental
-

4.2 Estrutura do Projeto

A organização do projeto foi dividida em módulos independentes:

password_cracker/

|

| └─ src/

| | └─ config.py

| | └─ oracle.py

```
|  ├── forca_bruta.py
|  ├── divisao_conquista.py
|  ├── programacao_dinamica.py
|  ├── genetico.py
|
|  ├── tests/
|  |   └── secret.txt
|
|  └── main.py
```

4.3 O Oráculo

O módulo `oracle.py` simulou um servidor de autenticação.

Ele forneceu três interfaces:

`check_full()`

Verifica a senha completa.

`check_partial()`

Permite validar apenas partes da senha.

`get_fitness()`

Retorna quantos caracteres estão corretos.

Além disso, o sistema contabilizou o número total de consultas realizadas pelos algoritmos.

5. Implementação dos Algoritmos

5.1 Força Bruta

A implementação utilizou `itertools.product()` para gerar todas as combinações possíveis.

O algoritmo testava sequencialmente cada senha utilizando:

```
oracle.check_full(attempt)
```

A execução encerrava ao encontrar a combinação correta.

5.2 Divisão e Conquista

A senha foi dividida em duas partes independentes:

- Primeira metade: posições 0–3
- Segunda metade: posições 4–7

Cada bloco foi resolvido separadamente utilizando:

```
oracle.check_partial(bloco, posicao)
```

Após encontrar ambas as partes, as strings foram concatenadas.

5.3 Programação Dinâmica

A solução foi construída caractere por caractere.

Um dicionário de memoização armazenou prefixos já avaliados:

```
memo[prefixo] = valido
```

Quando um prefixo inválido era identificado, todas as derivações possíveis eram descartadas.

5.4 Algoritmo Genético

A solução genética utilizou:

- População inicial aleatória
- Seleção dos melhores indivíduos
- Cruzamento entre indivíduos
- Mutação probabilística

A função fitness foi baseada em:

```
oracle.get_fitness(individuo)
```

O algoritmo encerrava quando o fitness máximo (8) era atingido.

6. Resultados Experimentais

6.1 Métricas Avaliadas

Foram analisadas as seguintes métricas:

1. Tempo total de execução
 2. Número de chamadas ao oráculo
 3. Consumo de memória
-

6.2 Comparação Qualitativa

Estratégia	Tempo	Hits no Oráculo	Memória	Garantia de Solução
Força Bruta	Muito Alto	Muito Alto	Baixo	Sim
Divisão e Conquista	Baixo	Moderado	Baixo	Sim
Programação Dinâmica	Muito Baixo	Muito Baixo	Moderado	Sim
Algoritmo Genético	Muito Baixo	Moderado	Moderado	Não determinístico

6.3 Discussão dos Resultados

A Força Bruta apresentou o pior desempenho devido ao crescimento exponencial do espaço de busca.

A Divisão e Conquista demonstrou redução drástica na complexidade ao quebrar a senha em subproblemas independentes.

A Programação Dinâmica mostrou grande eficiência ao evitar recomputações, reduzindo significativamente o número de consultas ao oráculo.

O Algoritmo Genético apresentou excelente desempenho prático, encontrando soluções rapidamente sem explorar o espaço completo de combinações. Entretanto, por se tratar de uma heurística, não existe garantia matemática de convergência em tempo finito.

7. Questões Analíticas

7.1 Viabilidade em Sistemas Reais

Em sistemas reais, servidores frequentemente bloqueiam o endereço IP após poucas tentativas incorretas.

Nesse cenário:

- Força Bruta torna-se inviável.
 - Divisão e Conquista depende de um sistema vulnerável que revele partes da senha.
 - Programação Dinâmica também depende de feedback parcial.
 - Algoritmos Genéticos poderiam continuar parcialmente viáveis caso existisse algum canal indireto de feedback.
-

7.2 O Custo do Oráculo

Na prática, sistemas reais não fornecem explicitamente um “fitness score”.

Entretanto, esse feedback pode surgir indiretamente por meio de:

- Tempo de resposta do servidor

- Consumo energético
- Diferenças em mensagens de erro
- Vazamentos de tempo (timing attacks)
- Side channels

Essas informações podem permitir inferências parciais sobre a senha.

7.3 Espaço versus Tempo

A Programação Dinâmica utilizou memória adicional para armazenar resultados intermediários.

Apesar do aumento no consumo de memória, a redução do número de consultas compensou amplamente o custo adicional.

Esse paradigma demonstrou claramente o clássico tradeoff entre espaço e tempo.

8. Conclusão

O presente trabalho demonstrou como diferentes paradigmas algorítmicos podem alterar drasticamente a eficiência computacional na resolução de um mesmo problema.

A Força Bruta mostrou-se extremamente custosa devido ao crescimento exponencial do espaço de busca.

A Divisão e Conquista evidenciou o impacto da decomposição do problema em subestruturas independentes.

A Programação Dinâmica reduziu recomputações utilizando memoização de resultados intermediários.

Os Algoritmos Genéticos demonstraram a eficiência de heurísticas evolutivas em problemas de busca complexos.

Além da comparação entre paradigmas, o projeto também evidenciou como sistemas que fornecem feedback excessivo podem comprometer significativamente a segurança de autenticação.

9. Trabalhos Futuros

Como possíveis extensões do projeto, destacam-se:

- Implementação de paralelismo na força bruta
- Uso de GPU para aceleração
- Algoritmos genéticos adaptativos
- Implementação de simulated annealing

- Análise estatística automatizada
 - Geração automática de gráficos experimentais
-

10. Referências

CORMEN, Thomas H. et al. Introduction to Algorithms. MIT Press.

KLEINBERG, Jon; TARDOS, Éva. Algorithm Design. Pearson.

RUSSELL, Stuart; NORVIG, Peter. Artificial Intelligence: A Modern Approach. Pearson.

STALLINGS, William. Cryptography and Network Security. Pearson.

TANENBAUM, Andrew. Computer Networks. Pearson.